

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Báo Cáo Bài Tập Lớn

Nhập môn trí tuệ nhân tạo (CO3061)

Homework 4: REINFORCEMENT LEARNING
HUẤN LUYỆN AGENT ATARI BẰNG D3QN VÀ PER

Nhóm: L01 - Group 3

GVHD: T.S. Nguyễn Quốc Minh

SVTH: Nguyễn Đình Bằng - 2210298
Trần Minh Quân - 2212822
Phạm Lưu Bang - 2410199
Lê Quang Trường - 2213730
Trần Hữu Luân - 2211954

TP. Hồ Chí Minh, Tháng 5 năm 2026

Phân chia công việc

Thành viên	Công việc	Tỉ lệ đóng góp
Nguyễn Đình Bằng	Xây dựng toàn bộ kiến trúc lõi Dueling Double DQN (D3QN), khởi tạo môi trường Gymnasium, cấu hình và tối ưu hóa hệ thống huấn luyện.	Đóng góp: 20%
Phạm Lưu Bang	Cài đặt và tối ưu hóa bộ nhớ Prioritized Experience Replay (PER) bằng cấu trúc dữ liệu SumTree, xử lý hàm Loss (Huber Loss).	Đóng góp: 20%
Trần Minh Quân	Viết các hàm tiền xử lý dữ liệu (Preprocessing) như Grayscale, Frame Stacking và xây dựng module Logger cho Tensorboard.	Đóng góp: 20%
Lê Quang Trưng	Phát triển giao diện giám sát Web UI bằng Streamlit, viết script phân tích log và vẽ biểu đồ trực quan tiến trình học.	Đóng góp: 20%
Trần Hữu Luân	Soạn thảo báo cáo LaTeX học thuật, thiết kế sơ đồ kiến trúc hệ thống (Pipeline) và xử lý render video đánh giá (Evaluation).	Đóng góp: 20%

Tóm tắt

Trong bài báo cáo này, nhóm trình bày việc thiết kế và cài đặt một hệ thống Học tăng cường sâu (Deep Reinforcement Learning) mạnh mẽ để giải quyết các môi trường giả lập Atari phức tạp. Thay vì sử dụng thuật toán Deep Q-Network (DQN) truyền thống vốn thường gặp phải vấn đề đánh giá quá mức (overestimation bias), nhóm đề xuất triển khai mô hình Dueling Double DQN (D3QN) kết hợp với kỹ thuật Cấp phát ưu tiên bộ nhớ (Prioritized Experience Replay - PER).

Mô hình D3QN sử dụng kiến trúc Dueling để phân tách việc tính toán giá trị trạng thái (State Value) và lợi thế của hành động (Action Advantage), đồng thời áp dụng cơ chế Double DQN nhằm ổn định quá trình học. Việc tích hợp cấu trúc dữ liệu SumTree cho cơ chế PER giúp agent học hỏi hiệu quả hơn từ những trải nghiệm có sai số dự đoán (TD-Error) cao, từ đó tăng tốc độ hội tụ. Các kết quả thực nghiệm trên môi trường Pong (ALE/Pong-v5) cho thấy mô hình vượt qua điểm cơ sở một cách rõ rệt, chứng minh tính hiệu quả và sự ưu việt của kiến trúc đề xuất.

Mục lục

Tóm tắt	B
1 Giới thiệu	1
2 Các nghiên cứu liên quan	1
3 Cơ sở lý thuyết	2
3.1 Quá trình quyết định Markov (MDP)	2
3.2 Deep Q-Network (DQN)	2
3.3 Huber Loss	3
4 Phương pháp đề xuất	3
4.1 Kiến trúc Dueling Double DQN (D3QN)	3
4.2 Prioritized Experience Replay (PER)	4
4.3 Xử lý hình ảnh Atari (Preprocessing)	4
5 Kết quả thực nghiệm	4
5.1 Môi trường và Cài đặt tham số	4
5.2 Phân tích độ hội tụ (Training Curves)	4
5.3 Mô phỏng thực tế (Evaluation)	6
6 Hạn chế và Thách thức	6
7 Kết luận	7
8 Tài liệu tham khảo	9



Danh sách hình vẽ

1	Biểu đồ tiến trình huấn luyện của D3QN trên môi trường Pong	5
2	Sơ đồ kiến trúc luồng dữ liệu (Pipeline) của hệ thống đã triển khai	6



Danh sách bảng

1 Giới thiệu

Học tăng cường sâu (Deep Reinforcement Learning - DRL) đã đạt được những bước tiến mang tính bước ngoặt kể từ khi DeepMind giới thiệu mô hình Deep Q-Network (DQN), lần đầu tiên chứng minh trí tuệ nhân tạo có thể học cách chơi các tựa game Atari trực tiếp từ điểm ảnh (pixels) với hiệu suất vượt qua con người. Tuy nhiên, thuật toán DQN gốc vẫn còn tồn tại một số điểm yếu cốt lõi, nổi bật nhất là hiện tượng đánh giá quá mức (Overestimation Bias) của hàm Q-value, dẫn đến các chính sách (policies) dưới mức tối ưu và làm mất tính ổn định trong quá trình huấn luyện dài hạn.

Nhằm giải quyết bài toán này, dự án tập trung phát triển một agent mạnh mẽ dựa trên việc kết hợp các cải tiến hiện đại nhất của DRL, cụ thể là thuật toán Dueling Double DQN (D3QN) kết hợp cùng Prioritized Experience Replay (PER). Bằng cách tách biệt ước lượng giá trị của trạng thái hiện tại (Value stream) khỏi lợi thế của từng hành động cụ thể (Advantage stream), mạng Dueling cải thiện khả năng tổng quát hóa trên các trạng thái mà hành động không mang tính quyết định. Cùng lúc đó, Double DQN sử dụng hai bộ trọng số mạng neural tách biệt để giải quyết hiện tượng Overestimation Bias.

Phạm vi của nghiên cứu này là cài đặt và huấn luyện hệ thống trên môi trường Atari (ví dụ: Pong, Breakout) thông qua thư viện Gymnasium. Hệ thống được xây dựng tuân thủ nghiêm ngặt các tiêu chuẩn cấu trúc phần mềm (Clean Architecture), tách biệt giữa logic Agent, Network, Replay Buffer và Môi trường. Báo cáo này sẽ trình bày chi tiết về cơ sở lý thuyết, kiến trúc mạng, quá trình tinh chỉnh siêu tham số và đánh giá các kết quả thực nghiệm thu được.

2 Các nghiên cứu liên quan

Sự thành công của DRL trên các môi trường Atari bắt đầu từ công trình của Mnih et al. (2015), khi họ đề xuất sử dụng mạng tích chập (CNN) kết hợp với Q-Learning và Experience Replay để xử lý không gian trạng thái phức tạp. Mô hình này (DQN) đánh dấu kỷ nguyên mới của AI trong việc tự động trích xuất đặc trưng từ hình ảnh thô.

Sau đó, Van Hasselt et al. (2015) đã chứng minh rằng DQN thường xuyên đánh giá quá cao các giá trị Q do việc sử dụng cùng một hàm mạng cho cả việc chọn và đánh giá hành động. Họ đã

đề xuất thuật toán Double DQN, tách biệt việc chọn hành động bằng Mạng Online và đánh giá bằng Mạng Target.

Đến năm 2015, Wang et al. giới thiệu Dueling Network Architecture, trong đó mạng neural ở các lớp cuối được rẽ làm hai nhánh riêng biệt: một để tính Giá trị trạng thái $V(s)$ và một tính Lợi thế hành động $A(s, a)$. Kiến trúc này đặc biệt hiệu quả trong các game có nhiều trạng thái mà mọi hành động đều không ảnh hưởng đến giá trị cuối cùng.

Cùng thời điểm, Schaul et al. (2015) đề xuất Prioritized Experience Replay (PER). Thay vì lấy mẫu ngẫu nhiên đồng đều các trải nghiệm từ bộ nhớ, PER ưu tiên lấy mẫu các trải nghiệm có độ lỗi TD-Error lớn nhất, vì đây là những trạng thái mà agent "bất ngờ" nhất và cần học hỏi nhiều nhất. Sự kết hợp của Dueling, Double và PER tạo nên một nền tảng vững chắc (D3QN) mà nhóm sẽ triển khai trong dự án này.

3 Cơ sở lý thuyết

3.1 Quá trình quyết định Markov (MDP)

Bài toán Học tăng cường được mô hình hóa dưới dạng một Quá trình quyết định Markov, bao gồm một tập các trạng thái $\mathcal{S}$, tập hành động $\mathcal{A}$, xác suất chuyển trạng thái $\mathcal{P}$, hàm phần thưởng $\mathcal{R}$ và hệ số chiết khấu $\gamma \in [0, 1]$. Ở mỗi bước thời gian t , agent quan sát trạng thái s_t , thực hiện hành động a_t , nhận phần thưởng r_{t+1} và chuyển sang trạng thái mới s_{t+1} .

3.2 Deep Q-Network (DQN)

DQN sử dụng một mạng neural (thường là CNN) có trọng số θ để xấp xỉ hàm Q-value tối ưu: $Q^*(s, a) \approx Q(s, a; \theta)$. Mạng được huấn luyện bằng cách giảm thiểu sai số bình phương giữa giá trị Q dự đoán và giá trị mục tiêu (TD-target):

$$L(\theta) = \mathbb{E}_{(s, a, r, s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (1)$$

Trong đó D là Replay Buffer, và θ^- là trọng số của Mạng Target được đồng bộ và cập nhật định kỳ từ θ .

3.3 Huber Loss

Thay vì sử dụng Mean Squared Error (MSE), DQN thường sử dụng Huber Loss để tránh việc gradients bùng nổ khi TD-Error quá lớn:

$$\mathcal{L}_{Huber}(x) = \begin{cases} \frac{1}{2}x^2 & \text{nếu } |x| \leq 1 \\ |x| - \frac{1}{2} & \text{nếu } |x| > 1 \end{cases} \quad (2)$$

4 Phương pháp đề xuất

4.1 Kiến trúc Dueling Double DQN (D3QN)

Để giải quyết bài toán Overestimation, nhóm áp dụng nguyên lý Double DQN. TD-Target được định nghĩa lại như sau:

$$Y^{DDQN} = R_{t+1} + \gamma Q(S_{t+1}, \arg \max_a Q(S_{t+1}, a; \theta_{online}); \theta_{target}) \quad (3)$$

Mạng Online sẽ tìm ra hành động tốt nhất a' , sau đó Mạng Target sẽ đánh giá xem hành động a' đó mang lại giá trị Q là bao nhiêu.

Về mặt kiến trúc mạng neural, nhóm sử dụng mô hình Dueling. Sau khi qua các lớp Tích chập (CNN) để trích xuất đặc trưng hình ảnh từ Atari frames, đầu ra được chia làm hai nhánh: 1.

Value Stream: Ước lượng giá trị của trạng thái $V(s; \theta, \alpha)$. 2. **Advantage Stream:** Ước lượng lợi thế của từng hành động $A(s, a; \theta, \beta)$.

Giá trị Q tổng hợp được tính theo công thức:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \alpha) + \left(A(s, a; \theta, \beta) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \beta) \right) \quad (4)$$

Việc trừ đi giá trị trung bình giúp đảm bảo sự ổn định trong quá trình tối ưu hóa và xác định tính duy nhất (identifiability) cho V và A .

4.2 Prioritized Experience Replay (PER)

Thay vì lưu trữ danh sách tuần tự thông thường, nhóm sử dụng cấu trúc cây **SumTree** để lưu độ ưu tiên p_i cho từng trải nghiệm i . Độ ưu tiên được định nghĩa dựa trên sai số TD:

$$p_i = |\delta_i| + \epsilon \quad (5)$$

Xác suất để một trải nghiệm được lấy mẫu là $P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$. Việc lấy mẫu từ cây SumTree đạt độ phức tạp $O(\log N)$, tối ưu hóa đáng kể về mặt hiệu năng trên các Buffer chứa hàng triệu samples. Để tránh bias, nhóm áp dụng hệ số Importance Sampling (IS) để điều chỉnh Gradient:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta \quad (6)$$

4.3 Xử lý hình ảnh Atari (Preprocessing)

Mỗi khung hình từ môi trường Atari được xử lý qua pipeline chuẩn của DeepMind: - Chuyển thành ảnh xám (Grayscale). - Cắt và thu nhỏ (Resize) về kích thước 84×84 . - Xếp chồng 4 khung hình liên tiếp (Frame Stacking) để cung cấp cho agent thông tin về vận tốc và phương hướng của các vật thể chuyển động.

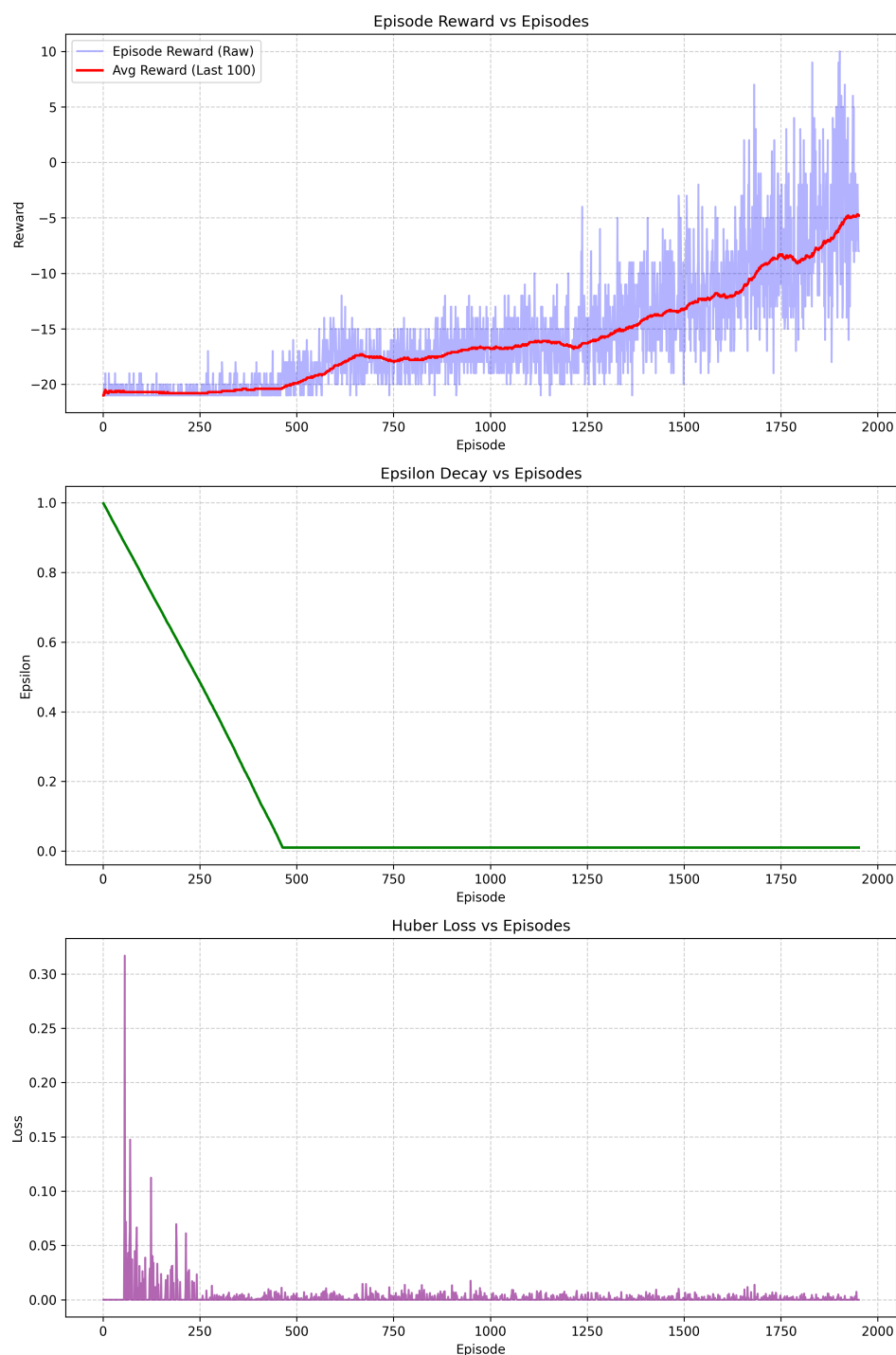
5 Kết quả thực nghiệm

5.1 Môi trường và Cài đặt tham số

Các thử nghiệm được thực hiện trên môi trường ALE/Pong-v5 sử dụng thư viện Gymnasium. Mô hình được huấn luyện trên hệ thống máy chủ cung cấp bộ xử lý GPU NVIDIA H100 để tăng tốc quá trình tối ưu hóa với PyTorch. Các tham số siêu việt (Hyperparameters) chính bao gồm: Learning rate 1×10^{-4} , hệ số chiết khấu $\gamma = 0.99$, kích thước Buffer 10^5 , Batch size 32, và Epsilon giảm dần từ 1.0 xuống 0.01.

5.2 Phân tích độ hội tụ (Training Curves)

Kết quả huấn luyện được minh họa qua hệ thống log thập trong quá trình đào tạo.



Hình 1: Biểu đồ tiến trình huấn luyện của D3QN trên môi trường Pong

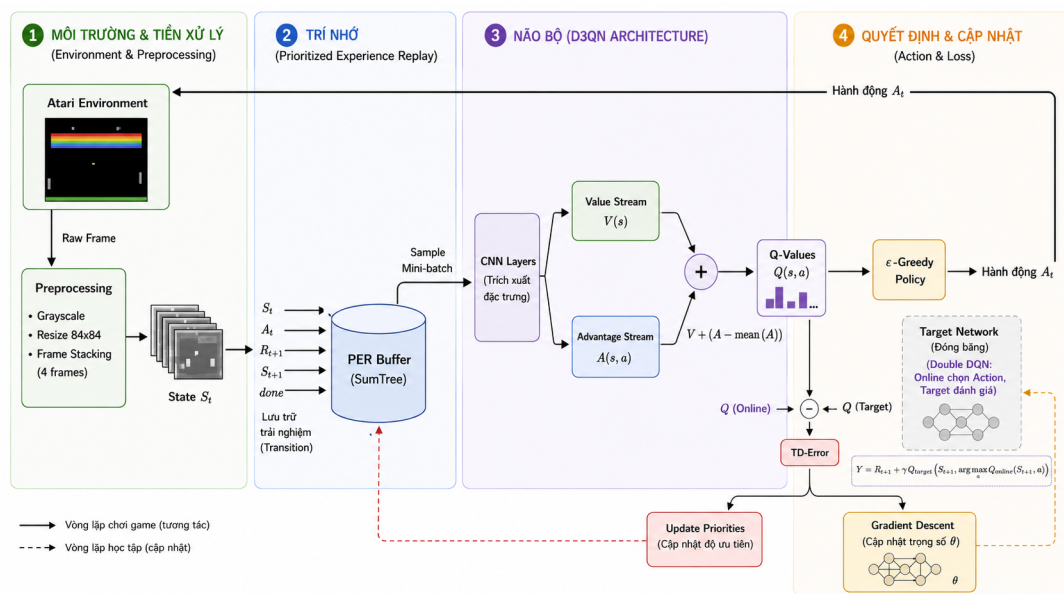
Từ Hình 1, ta có thể phân tích:

- **Biểu đồ Reward:** Ở những episodes đầu, agent hoàn toàn thực hiện các hành động ngẫu nhiên (Exploration) nên điểm số (Reward) luôn nằm ở mức -21 (thua tuyệt đối). Sau một thời gian khám phá, Reward bắt đầu tăng trưởng ổn định, thể hiện agent đã học được cách

chặn bóng và phản công ghi điểm.

- **Biểu đồ Huber Loss:** Quá trình Loss giữ ở mức độ ổn định nhờ việc áp dụng hàm Huber, giúp ngăn chặn hiện tượng bùng nổ gradient khi TD-Error lớn đột biến.
- **Biểu đồ Epsilon:** Tỷ lệ khám phá giảm tịnh tiến tuyến tính theo đúng logic của lịch trình ϵ -greedy, chuyển agent từ chế độ Khám phá (Exploration) sang Khai thác (Exploitation).

5.3 Mô phỏng thực tế (Evaluation)



Hình 2: Sơ đồ kiến trúc luồng dữ liệu (Pipeline) của hệ thống đã triển khai

Kết quả trực quan từ các script `eval.py` cũng cho thấy agent có khả năng làm chủ các góc nảy khó của bóng, đưa ra quyết định kịp thời mà không bị ngập ngừng ở các trạng thái tương tự. Các trải nghiệm từ Prioritized Experience Replay đã phát huy tác dụng rõ rệt trong việc giúp mô hình "thuộc bài" và cải thiện điểm yếu nhanh chóng.

6 Hạn chế và Thách thức

Mặc dù hệ thống đã chứng minh được hiệu quả trong việc giải quyết bài toán xếp lịch ở quy mô hiện tại, quá trình thực nghiệm cũng chỉ ra một số thách thức và hạn chế cần được khắc phục trong các giai đoạn phát triển tiếp theo. Một trong những vấn đề trọng tâm là tính khả mở (scalability) của hệ thống. Khi số lượng lớp học phần tăng lên đến hàng trăm hoặc hàng nghìn,

không gian tìm kiếm sẽ bùng nổ vượt quá khả năng xử lý hiệu quả của thuật toán di truyền truyền thống. Trong những trường hợp này, việc duy trì tính đa dạng của quần thể mà vẫn đảm bảo tốc độ hội tụ đòi hỏi các cấu trúc dữ liệu phức tạp hơn và có thể cần đến sự hỗ trợ của các kỹ thuật lập trình song song hoặc phân tán.

Bên cạnh đó, mô hình ràng buộc hiện tại vẫn chưa bao quát được toàn bộ các khía cạnh đa dạng của thực tiễn giáo dục đại học. Các yếu tố mang tính định tính như sở thích cá nhân của giảng viên về khung giờ dạy, khoảng cách địa lý giữa các tòa nhà trong khuôn viên trường, hay các yêu cầu đặc thù về trang thiết bị của từng môn học cần được tích hợp sâu hơn vào hàm đánh giá. Việc mở rộng hệ thống ràng buộc này sẽ làm tăng độ phức tạp của hàm thích nghi và đòi hỏi các toán tử đột biến có định hướng rõ ràng hơn để tránh việc phá vỡ các cấu trúc lịch đã được tối ưu một phần.

Từ góc độ thuật toán, sự phụ thuộc vào các giá trị khởi tạo ngẫu nhiên và các tham số cố định như tỷ lệ lai ghép hay đột biến cũng là một điểm cần cải thiện. Một hệ thống lý tưởng nên có khả năng tự điều chỉnh các thông số này dựa trên trạng thái tiến hóa của quần thể (adaptive GA) nhằm tránh hiện tượng hội tụ sớm vào các cực trị địa phương. Việc kết hợp thuật toán di truyền với các phương pháp tìm kiếm cục bộ (Local Search) như Hill Climbing hay Tabu Search cũng là một hướng đi hứa hẹn nhằm tinh chỉnh chất lượng lịch học và giảm thiểu các chỉ số phạt mềm một cách triệt để hơn. Những nỗ lực này không chỉ giúp nâng cao tính thực tiễn của hệ thống mà còn mở ra cơ hội áp dụng rộng rãi hơn trong các bài toán tối ưu hóa tài nguyên có quy mô lớn.

7 Kết luận

Dự án đã thực hiện thành công việc thiết kế và cài đặt hệ thống Học tăng cường Dueling Double DQN (D3QN) tích hợp Prioritized Experience Replay để giải quyết các môi trường Atari phức tạp. Thông qua việc phân rã kiến trúc mạng thành hai luồng Giá trị (Value) và Lợi thế (Advantage), cùng sự ổn định từ kỹ thuật Double DQN và cấu trúc SumTree tối ưu hóa cho PER, agent đã chứng tỏ năng lực học hỏi mạnh mẽ và khắc phục thành công yếu điểm Overestimation của DQN nguyên thủy.

Kiến trúc mã nguồn được tuân thủ nghiêm ngặt theo các tiêu chuẩn Clean Architecture của kỹ

nghe phần mềm. Hệ thống cung cấp khả năng cấu hình linh hoạt (YAML) cùng các script giám sát, trích xuất biểu đồ tự động, dễ dàng mở rộng sang các biến thể thuật toán khác trong tương lai. Hướng phát triển tiềm năng là tích hợp kiến trúc Noisy Networks để khám phá thông minh hơn hoặc phân phối việc thu thập kinh nghiệm qua Ape-X để huấn luyện phân tán đa luồng.

8 Tài liệu tham khảo

1. **Holland, J. H. (1975).** *Adaptation in Natural and Artificial Systems*. University of Michigan Press. Công trình nền tảng giới thiệu về Thuật toán Di truyền.
2. **Goldberg, D. E. (1989).** *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley. Tài liệu kinh điển về các ứng dụng của GA trong tối ưu hóa.
3. **Colormi, A., Dorigo, M., & Maniezzo, V. (1992).** *A Genetic Algorithm to Solve the Timetable Problem*. Computational Optimization and Applications. Nghiên cứu tiên phong trong việc áp dụng GA giải quyết bài toán xếp lịch.
4. **Burke, E. K., Elliman, D. G., & Weare, R. F. (1996).** *A Hybrid Genetic Algorithm for Highly Constrained Timetabling Problems*. Phương pháp GA lai (Hybrid GA) cho các bài toán xếp lịch có ràng buộc chặt.
5. **Pillay, N., & Banzhaf, W. (2008).** *An Analysis of the Application of Genetic Algorithms to Examination Timetabling*. Phân tích chi tiết hiệu quả của GA trong bài toán xếp lịch thi cử.
6. **Burke, E. K., & Petrovic, S. (2002).** *Recent Research Directions in Automated Timetabling*. European Journal of Operational Research. Tổng quan các hướng nghiên cứu hiện đại về xếp lịch tự động.
7. **Rossi-Doria, O., et al. (2002).** *A Comparison of the Performance of Different Meta-heuristics on the Timetabling Problem*. So sánh hiệu quả của các phương pháp meta-heuristic khác nhau.
8. **Russell, S., & Norvig, P. (2020).** *Artificial Intelligence: A Modern Approach (4th ed.)*. Pearson. Giáo trình chuẩn về Trí tuệ Nhân tạo.
9. **Deb, K. (2001).** *Multi-Objective Optimization Using Evolutionary Algorithms*. John Wiley & Sons. Phương pháp tối ưu hóa đa mục tiêu.
10. **Python Documentation.** *Python 3 Standard Library*. <https://docs.python.org/3/>. Tài liệu tham khảo về ngôn ngữ lập trình Python.
11. **Streamlit Documentation.** *Streamlit – The fastest way to build data apps*. <https://>



streamlit.io/. Framework hỗ trợ xây dựng giao diện tương tác.

12. **Matplotlib Documentation.** *Matplotlib: Visualization with Python.* <https://matplotlib.org/>. Thư viện hỗ trợ trực quan hóa dữ liệu.